

XiaoZhi 语音助手

基于 ESP32-S3-Touch-LCD-3.49 v2 的智能语音交互设备

ESP-IDF v6.0.2 · LVGL 9 · MQTT/WebSocket · Opus 编码

项目概述

🎯 项目定位

XiaoZhi 是一个开源的 ESP32 语音助手固件，支持多种芯片、显示屏、音频设备和网络传输方式，通过 WiFi 连接云端 AI 实现语音对话。

🏗️ 架构特点

板级抽象设计，一套核心代码支持多种硬件；通过 OTA 服务器配置协议（MQTT/WebSocket），设备激活后即插即用。

🌐 服务集成

连接云端 AI 服务，支持语音识别（ASR）、大语言模型对话（LLM）、语音合成（TTS），以及 MCP 工具调用。

🔧 本项目定制

基于 Waveshare ESP32-S3-Touch-LCD-3.49 v2 开发板，定制了屏幕旋转、电池监控、IMU 姿态唤醒、主题切换等功能。

硬件平台

核心规格

- 芯片：ESP32-S3 双核 240MHz
- Flash：16MB
- PSRAM：8MB OCT PSRAM
- WiFi：802.11 b/g/n
- 蓝牙：BLE 4.2

显示触摸

- 屏幕：3.49" 彩色 LCD
- 分辨率：172×640（竖屏）
- 驱动：AXS15231B（QSPI）
- 触摸：FT5x06（I2C）
- 接口：QSPI + I2C

音频

- 输出 DAC：ES8311
- 输入 ADC：ES7210（4通道）
- 扬声器：板载
- 麦克风：双 MEMS

传感器与电源

- IMU：
  QMI8658（加速度+陀螺仪）
- 电池：内置锂电池 + ADC 监控
- 分压比：3:1（200kΩ/100kΩ）
- 按钮：BOOT + PWR 双键

已实现功能概览

显示系统

- 屏幕 270° 旋转
- 双击切换主题
- LVGL 9 UI 框架
- 状态栏 + 聊天气泡

电源管理

- 电池电量百分比
- 充电/放电状态
- 低电量弹窗提醒
- ADC 实时采样

姿态唤醒

- QMI8658 IMU 读取
- 抬手触发聆听
- 0.1g 灵敏度
- 5秒冷却防误触

语音交互

- 自定义唤醒词"你好小威"
- MultiNet 拼音识别
- Opus 音频编码
- VAD 自动停止

网络通信

- MQTT 协议（默认）
- WebSocket 协议
- UDP 音频传输
- AES-CTR 加密

性能优化

- 跳过 OTA 版本检查
- 停用固件自动升级
- NVS 配置缓存
- 快速启动

显示系统：屏幕旋转

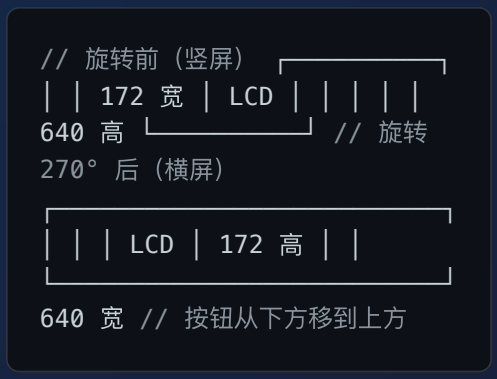
🎯 需求

原厂屏幕为 172×640 竖屏，物理按钮在下方会遮挡 UI。需要旋转为横屏显示，按钮朝上。

⚙️ 实现方案

- LVGL 软件旋转 270°
- 逻辑分辨率变为 640×172 横屏
- `lv_display_set_rotation(LV_DISPLAY_ROTATION_270)`
- flush 回调用 `lv_draw_sw_rotate` 旋转缓冲区

📐 坐标变换



UI 使用 **LV_HOR_RES/LV_VER_RES** 动态宏，自动适配旋转后的分辨率。

显示系统：主题切换

🎨 双主题系统

主题	背景色	文字色
Light	白色 0xFFFFFF	黑色 0x000000
Dark	黑色 0x000000	白色 0xFFFFFF

触摸交互

- **双击**屏幕任意位置切换
- LVGL 事件冒泡机制
- 主题持久化到 NVS
- 重启保持上次选择

🔄 工作流程

```
// CustomLcdDisplay::SetupUI() void SetupUI() override { LcdDisplay::SetupUI(); // 父类初始化 // 给子对象添加事件冒泡 for (child : container_->children) { lv_obj_add_flag(child, LV_OBJ_FLAG_EVENT_BUBBLE); } // 注册双击事件 lv_obj_add_event_cb(container_, OnThemeSwitchEvent, LV_EVENT_DOUBLE_CLICKED, this); } // 切换 light ↔ dark void OnThemeSwitchEvent(e) { next = (current == "light") ? "dark" : "light"; SetTheme(GetTheme(next)); // 重绘+保存 }
```

电池电量监控

硬件电路

- ADC1 通道 3（GPIO4）
- 分压电阻：200kΩ / 100kΩ
- 分压比 3:1
- 测量范围：0-3.3V × 3 = 0-9.9V

软件实现

- `AdcBatteryMonitor` 类
- 1Hz 采样频率
- 滑动平均滤波
- 电量百分比映射

状态输出

```
// GetBatteryLevel() 返回 struct BatteryInfo { int8_t level; // 0-100 百分比 bool charging; // 是否充电中 bool discharging; // 是否放电中 }; // 实测数据 Battery: level=96%, charging=0, discharging=1
```

UI 集成

状态栏自动显示电池图标，低电量（<20%）弹出提醒窗口。

IMU 姿态唤醒

🎯 功能描述

通过 IMU 检测设备姿态变化，**抬手或翻转设备**自动进入聆听模式，无需按按钮或说唤醒词。

⚙️ 实现参数

参数	值
传感器	QMI8658
接口	I2C（音频总线共用）
采样率	5Hz
触发阈值	0.1g（极灵敏）
冷却时间	5 秒

🔄 工作流程

IMU 采样加速度

↓

计算合成加速度

↓

变化量 > 0.1g?

↓ 是

检查冷却时间

↓ 通过

ToggleChatState() 进入聆听

使用 `ToggleChatState()` 而非 `StartListening()`，确保 AutoStop 模式说完话自动退出。

语音唤醒：自定义唤醒词

🎙️ 唤醒词配置

"你好小威"

拼音：ni hao xiao wei

⚙️ 技术方案

- 模式：MultiNet 自定义唤醒词
- 模型：ESP-SR Multinet
- 阈值：20（1-99，越小越灵敏）
- 配置：NVS 持久化

📋 AFE vs MultiNet 对比

特性	AFE WakeNet	MultiNet
唤醒词	固定（你好小智等）	任意拼音
识别精度	高	中等
打断AI	✅ 支持	❌ 不支持
资源占用	高	低

本项目选择 MultiNet 以支持自定义唤醒词"你好小威"，代价是不能打断 AI 回复。

音频系统架构

上行：麦克风 → 服务器



下行：服务器 → 扬声器



Opus 16kHz/16bit　VAD 语音检测　AEC 回声消除　多队列缓冲

网络协议架构

📡 MQTT 协议（默认）

控制通道： MQTT publish/subscribe
音频通道： UDP + AES-CTR 加密
默认端口： 8883（TLS）
心跳： 90 秒 ping
重连： 60 秒自动重连

MQTT 用 UDP 传输音频以降低延迟，控制指令用 MQTT 保证可靠性。

🌐 WebSocket 协议

控制通道： WebSocket text frame
音频通道： WebSocket binary frame
默认端口： 443（TLS）
单连接： 文本+音频共用

WebSocket 单通道更简单，但音频延迟略高于 UDP。

🔄 协议选择流程

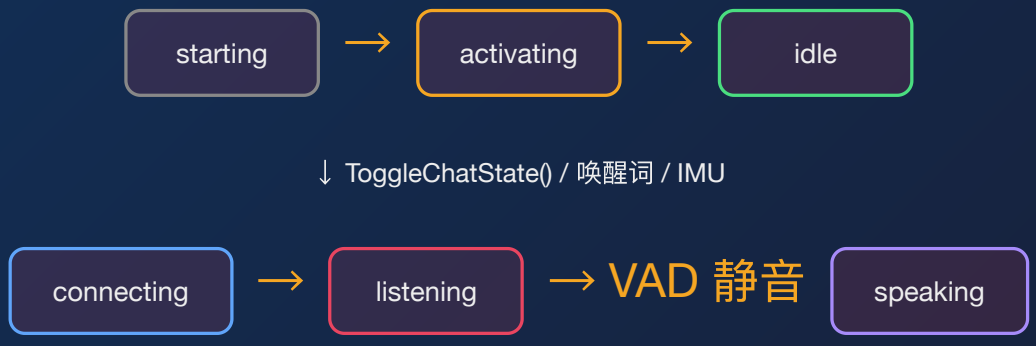


数据传输路径

```
// 设备 → 服务器：控制指令（JSON） { "session_id": "xxx", "type": "listen", "state": "start", "mode": "auto" // auto/manual/realtime } // 服务器 → 设备：TTS 回复 { "type": "tts", "text": "你好，我是小威", "audio_url": "https://.../xxx.mp3" } // 服务器 → 设备：MCP 工具调用 { "type": "mcp", "method": "tools/call", "params": {"name": "self.reboot"} } // 设备 → 服务器：音频流（Opus 编码后） [二进制数据：Opus 帧 + 序列号 + AES-CTR 加密]
```

JSON 控制指令 | Opus 音频流 | AES-CTR 加密 | UDP 低延迟

设备状态机



状态转换通过 `SetDeviceState()`，所有 UI 更新通过事件回调异步执行。

OTA 与设备激活

📋 激活流程

1. 设备连接 WiFi
2. 请求 `api.tenclass.net/xiaozhi/ota/`
3. 服务器返回配置：
   - MQTT/WebSocket 地址
   - 激活码（首次）
   - 服务器时间
   - 固件版本信息
4. 写入 NVS 持久化
5. 显示激活码给用户
6. 用户在 App 输入激活码
7. 激活完成，进入 idle

⚡ 性能优化

跳过版本检查（已实现）

检测 NVS 已有 MQTT/WebSocket 配置时，直接跳过 OTA 版本检查循环，大幅加速启动。

停用固件升级（已实现）

注释掉 `UpgradeFirmware()` 调用，防止服务器远程推送固件升级，保留配置获取和激活功能。

MCP 工具调用

设备内置 MCP 服务器，AI 可通过 JSON-RPC 调用设备端工具：

📱 系统工具

- `self.get_device_status`
- `self.reboot`

🔊 音频工具

- `audio.set_volume`
- `audio.play_sound`

🖥️ 显示工具

- `screen.set_brightness`
- `theme.set_theme`

🔄 调用流程

用户说"重启" → AI 解析意图 → 发送 MCP 请求 → 设备执行 Reboot()

技术亮点

🎨 LVGL 软件旋转

不依赖硬件旋转，通过 `lv_draw_sw_rotate` 在 flush 回调中实时旋转帧缓冲区，支持任意角度。

🤸 IMU 姿态触发

复用音频 I2C 总线连接 QMI8658，5Hz 采样，0.1g 阈值，无需额外 GPIO。

⚡ 快速启动

NVS 配置缓存，跳过 OTA 版本检查，已激活设备秒级进入待机状态。

🔒 加密音频

UDP 音频流使用 AES-CTR 加密，PSA 加密 API，防止音频被窃听。

🎭 双主题系统

Light/Dark 主题完整切换，LVGL 事件冒泡实现双击切换，NVS 持久化。

🔋 电池集成

ADC 分压采样，状态栏自动显示电量，低电量弹窗保护。

项目数据统计

9+ 已实现功能	**2** 网络协议	**2** 显示主题
3 唤醒方式	**0.1g** IMU 灵敏度	**270°** 屏幕旋转

ESP32-S3 · ESP-IDF v6.0.2 · LVGL 9 · Opus 编码 · MQTT · WebSocket · AES-CTR · QMI8658

未来展望

📦 SD 卡存储

移植原厂 demo 的 SD 卡功能，用于离线语音提示音、对话日志、配置备份。

⏰ RTC 时钟

集成 PCF85063 RTC 芯片，断电保持时间，开机快速同步系统时间。

💤 深度睡眠

电量低自动进入深度睡眠，延长电池寿命；物理按钮唤醒。

⚙️ UI 设置菜单

触摸屏设置界面：重启、休眠、音量、主题、唤醒词灵敏度等。

🎤 AFE 模式

切换到 AFE WakeNet，支持打断 AI 回复（需放弃自定义唤醒词）。

🌡️ 温控保护

读取芯片温度，过高自动降频或重启，保护硬件。



谢谢

XiaoZhi 语音助手项目

基于 ESP32-S3-Touch-LCD-3.49 v2

ESP-IDF v6.0.2 · LVGL 9 · MQTT · Opus